Sistema Robotico de Grua

Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio
Electrónica digital II

Índice

1. Identificación del proyecto **2**

2. Resumen **2**

3. Descripción del hardware **2**

4. Descripción del software **4**

5. Estructura del código **6**

6. Explicación de funciones principales **6**

7. Comunicación (si aplica) **7**

8. Interfaz de usuario (si aplica) **8**

9. Procedimiento de prueba **8**

10. Manejo de errores y seguridad **9**

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema Robotico de Grua
- **Integrantes del equipo: Raquel Arias García Diana Vanessa Guerra López Claudia Elena Durango Cossio**

2. Resumen

El presente proyecto se enfoca en el diseño y prueba de grúa robótica de sobremesa. Utilizando servomotores que permite 2 grados de libertad (GDL), ejecutados a través de un microcontrolador ESP32. El sistema permite controlar los servomotores de 2 diferentes modos: El modo manual controlado atreves potenciómetros, mantiene encendido un led de color verde. Cuando se activa el segundo se hace mediante interrupciones al activar uno de los pulsadores; dado que, este modo cuenta con 2 submodos dependerá de cual se presione pues uno tiene secuencia preestablecida antes de llegar al origen. Mientras que el otro, llegará a la posición (0, 0) de los servomotores. En ambos modos se apaga el led verde al tiempo que se activa led rojo indicando que se ha activado una interrupción; a su vez sonará un buzzer alertando una alarma.

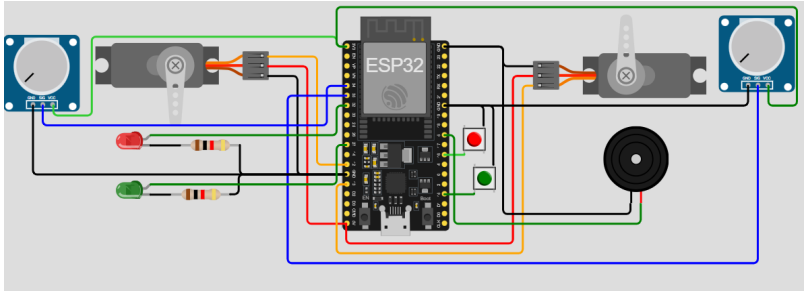
El sistema incluye control antirrebote por software en el caso de los botones, manejo de LEDs mediante registros, sistema de PWM para para controlar los servomotores y el buzzer, mientras que los potenciómetros se leen con ADC (señal analógica) en el modo manual.

3. Descripción del hardware

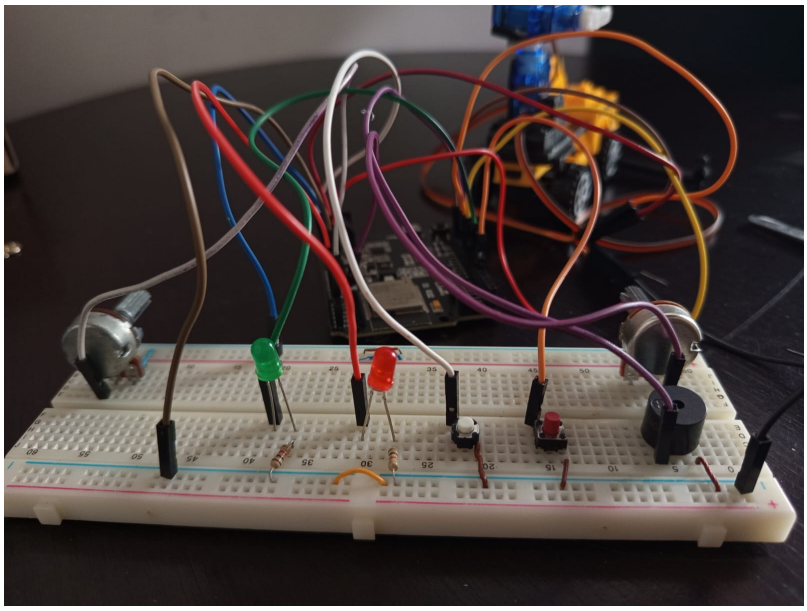
- **ESP32** (modelo con ADC y pines GPIO)
- **LED 1:** Pin 27
- **LED 2:** Pin 4
- **Buzzer:** Pin 5 18
- **Boton Modo Reset:** Pin 0
- **Boton Modo Automático:** Pin 16
- **Servomotor Base:** Pin 13

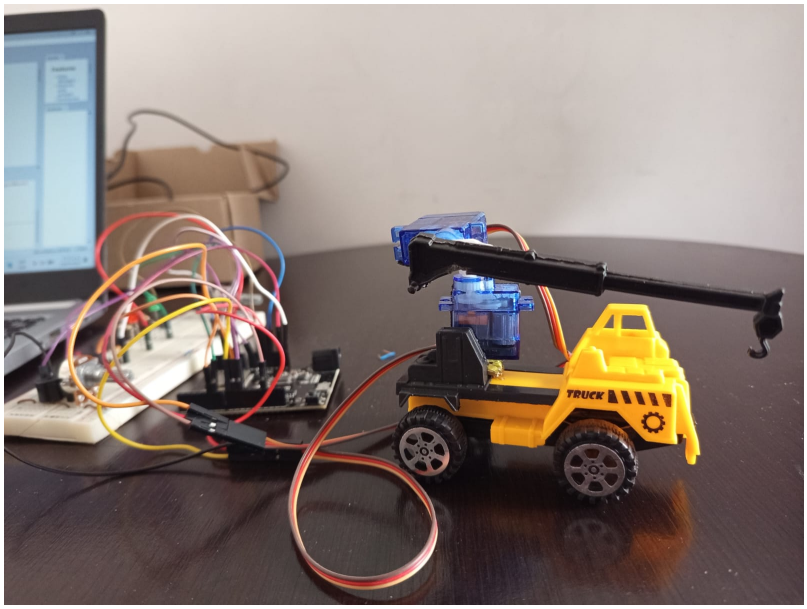
- **Servomotor Brazo:** Pin 12
- **Potenciómetro Base:** Pin 34
- **Potenciómetro Brazo:** Pin 32
- **Fuente de alimentación:** 3.3V (USB o regulada) y 5V para cada servomotor

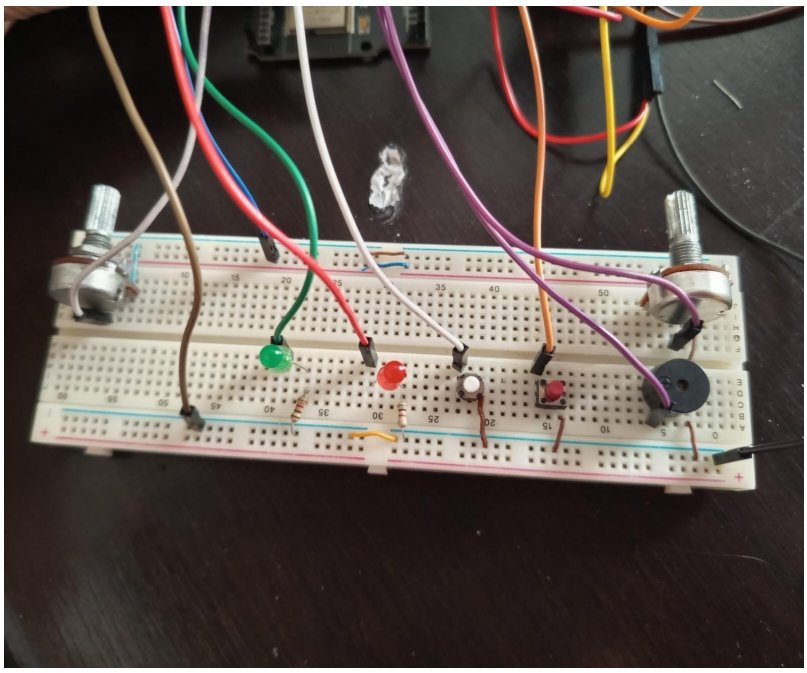
Diagrama de conexión:



Montaje en fisico:







4. Descripción del software

- **Lenguaje usado:** MicroPython

- **Librerías:** `machine`, `time`
- **IDE:** Thonny

Flujo general del programa:

1. Definición de pines de entradas y salidas (Pulsadores, leds y buzzer)
2. Configuración de servomotores mediante PWM
3. Configuración de potenciómetros mediante ADC
4. Configuración de interrupciones (Modo Automático y Reset) mediante botones.
5. Inicialización de variables del sistema.
6. Mostrar en pantalla "Sistema listo".
7. Estado inicial: modo manual, permite maniobrar el brazo y base de la grúa manualmente mediante los potenciómetros.
8. Ejecución del ciclo principal (máquina de estados), evalúa estado actual en bucle infinito.
9. Lectura de potenciómetro.
10. Conversión de valores analógicos a ángulos
11. Control directo de los servomotores
12. Encendido de LED verde indicador de modo manual
13. Monitoreo de interrupciones
14. Interrupción Modo Reset.
15. Activación de buzzer y LED de alerta
16. Movimiento suave de los servos hacia la posición inicial (0°, 0°)
17. Cambio de estado a ESPERA
18. Sistema en reposo
19. Verificación de que los potenciómetros estén en posición inicial
20. Cuando se cumple la condición Reset vuelve a modo MANUAL

5. Estructura del código

- Archivo principal: `main.py`
- Importación de librerías (Pin, PWM, ADC, time).
- Configuración del hardware.
- Servomotores mediante PWM.
- Potenciómetros mediante ADC.
- LEDs como salidas digitales.
- Buzzer mediante PWM.
- Botones como entradas con resistencias pull-up
- Inicialización de variables globales para controlar el estado del sistema : ESTADO (Máquina de estados), ultimo-evento (Antirrbote o Debounce), DEBOUNCE (Tiempo mínimo de pulsaciones válidas), pos-base y pos-braz (guarda la posición actual del servo de la base y brazo respectivamente (en grados)).
- Definición de funciones del sistema.
- Configuración de interrupciones mediante pulsadores (Incluye control antirrebote por software).
- Botón de reset activa modo retorno
- Botón automático activa modo secuencia
- Código lineal dentro de un ciclo infinito con lectura y decisión en tiempo real, ejecuta la función correspondiente a cada modo: Manual, retorno, secuencia y espera.

6. Explicación de funciones principales

- `angulo-a-duty(angulo)`: Conversión de ángulo (0°–180°) en una señal PWM (duty cycle) ideal para el control del servomotor.
- `mover(servo, angulo)`: Posiciona directamente un servomotor en el ángulo indicado, utilizando la conversión a PWM.

- `mover-suave(servo, actual, destino, paso=2)`: Realiza un movimiento progresivo del servo desde una posición inicial hasta una final, evitando cambios bruscos.
- `leer-pots()`: Lee los valores de los potenciómetros mediante el ADC y los convierte a ángulos entre 0° y 180°.
- `alarma(on)`: Activa o desactiva el buzzer y el LED de modo automático, funcionando como indicador visual y sonoro.
- `irq-reset(pin)`: Función de interrupción del botón de reset. Cambia el estado del sistema a RETORNO, aplicando antirrebote.
- `irq-auto(pin)`: Función de interrupción del botón automático. Cambia el estado a SECUENCIA, también con control antirrebote.
- `modo-manual()`: Permite controlar los servomotores en tiempo real mediante los potenciómetros. Mantiene encendido el LED de modo manual.
- `modo-retorno()`: Lleva la grúa a la posición inicial (0°, 0°) de forma suave, activando señales de alerta. Luego pasa a modo espera.
- `modo-secuencia()`: Ejecuta una serie de movimientos predefinidos y luego regresa a la posición inicial del sistema.
- `modo-espera()`: Mantiene el sistema en reposo hasta que los potenciómetros estén en posición inicial, permitiendo volver al modo manual.

7. Comunicación (si aplica)

Este proyecto utiliza comunicación interna del microcontrolador

1. `ADC (Conversión Analógico-Digital)`: Permite leer los potenciómetros, permitiendo convertir señales analógicas en valores digitales procesables.
2. `PWM (Modulación por Ancho de Pulso)`: Ideal para controlar los servomotores y el buzzer, regulando su posición y frecuencia de operación respectivamente.
3. `GPIO (Entradas/Salidas digitales)`: Necesarios par el control de LEDs y la lectura de botones mediante niveles lógicos.

4. `Interrupciones externas`: Implementadas en los botones para permitir cambios de estado inmediatos sin necesidad de consulta constante (polling).

8. Interfaz de usuario (si aplica)

El sistema posee una interfaz de usuario física y básica, compuesta por: Potenciómetros que ayudan al usuario controlar manualmente la posición de los servomotores. 2 Botones (Botón de reset: Activa el modo retorno y botón automático que inicia la secuencia predefinida). LEDs indicadores; el LED verde y LED rojo corresponde modo manual y modos automáticos respectivamente, El Buzzer proporciona retroalimentación auditiva durante la ejecución de acciones automáticas. Terminal serial (consola) Presenta mensajes informativos como “Sistema listo”, facilitando la supervisión del sistema. La interacción es directa y en tiempo real, sin interfaz gráfica.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Cargar el código en el ESP32.
3. Abrir consola para visualizar "Sistema listoïndicador de que el modo manual ha iniciado.
4. Prueba de modo manua: Verificar que los potenciometros muevan el servomotor correspondiente; al tiempo que alumbra el led verde.
5. Prueba de interrupciones: Verificar que ambos pulsadores ejecuten la interrupción al ser presionados individualmente, sin rebote y en cualquier momento siempre que el sistema esté en su modo normal. Mientras la una de las 2 interrupciones está activa, debe alumbrar unicamente el led rojo y sonar el buzzer.
6. Prueba de modo retorno: Confirmar el movimiento suave hacia la posición inicial (0°, 0°), verificación de activación de buzzer y LED rojo y validación del cambio al estado de espera.
7. Prueba de modo secuencia: Ejecución completa de la secuencia predefinida, verificación de cada posición intermedia y confirmación del retorno a la posición inicial.

10. Manejo de errores y seguridad

- `Antirrebote`: Basado en sofware por tiempo mediante (ticks-ms), previene falsas pulsaciones.
- `Validacion de estados`: evita que se se ejecuten acciones inesperadas en cada modo.
- `Control de rangos en servomotores`: Previene daños mecanicos en los sevomotores porque estableece rangos de giro de 0 a 180°.
- `Espera`: luego del modo retorno, no inicia instatanemente al modo manual, sino que espera a que los potenciometros esten en su posición inicial, esto ptebiee posibles daños con el servomotor.